TAREA 2 · BASES DE DATOS NOSQL

EventFlow

Venta de entradas con microservicios, bases políglotas, SAGA orquestada y Chain of Responsibility

FastAPI MongoDB Redis PostgreSQL Docker Compose

Pre-defensa · 5 de noviembre de 2026

Repositorio: TAREA 2/

AGENDA

Qué vamos a ver

01 El problema y los requisitos

02 Arquitectura

03 Selección de bases de datos

04 Estrategia de tres bases

05 Modelado: embebido vs referencia

06 Consistencia por operación

07 Lecturas AP con caché

08 Chain of Responsibility

09 SAGA orquestada y compensaciones

10 Atomicidad: no hay doble venta

11 Idempotencia y resiliencia

12 Auditoría: event log + CQRS

13 Privacidad: export anonimizado

14 Resultados, límites y demo

Dos exigencias opuestas en el mismo sistema

Lecturas

Catálogo y perfiles

- Muchas consultas de usuarios y eventos
- Rápidas y escalables
- Prioridad: **disponibilidad** y tolerancia a particiones
- Se acepta consistencia eventual

Escrituras

Reservas y pagos

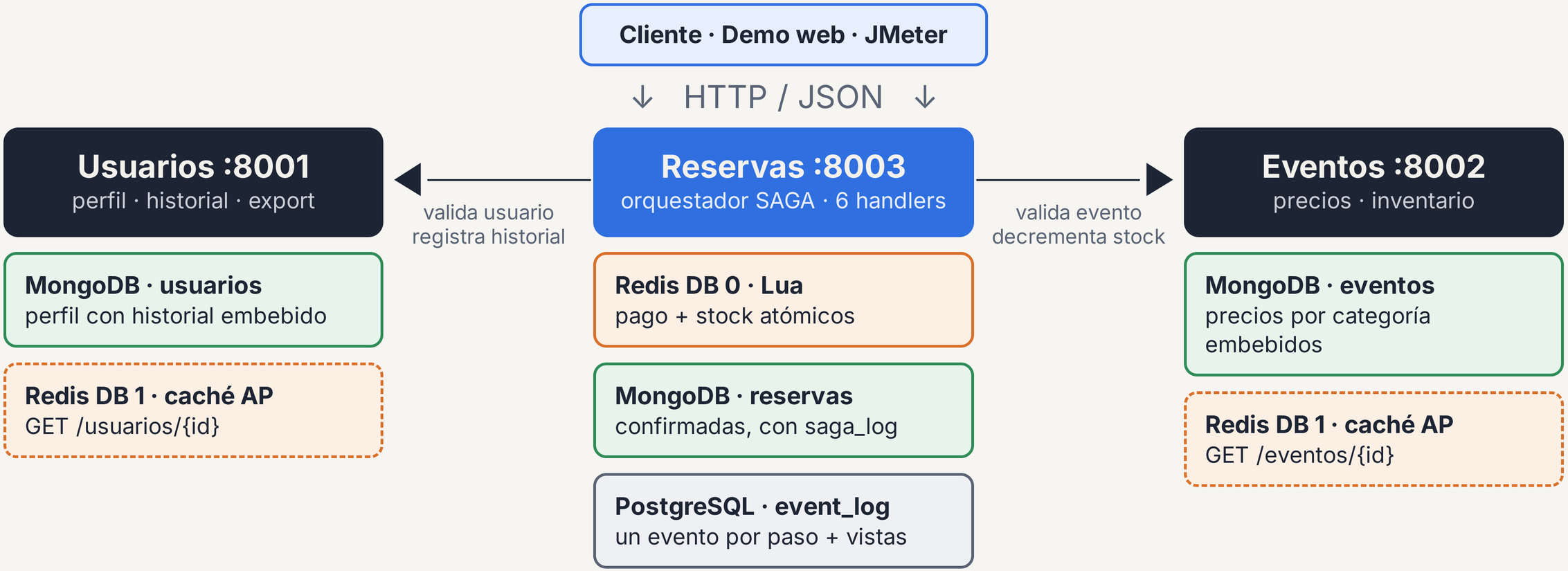
- Muchos compradores a la vez sobre el mismo stock
- Prioridad: **consistencia fuerte**
- Nunca vender de más ni cobrar dos veces
- Transacción que cruza servicios y bases

Requisitos

Del enunciado

- Al menos 2 bases NoSQL
- SAGA con orquestación
- Chain of Responsibility
- Docker Compose
- Opcionales: JMeter, Event Sourcing, CQRS, anonimización

Tres microservicios, tres bases



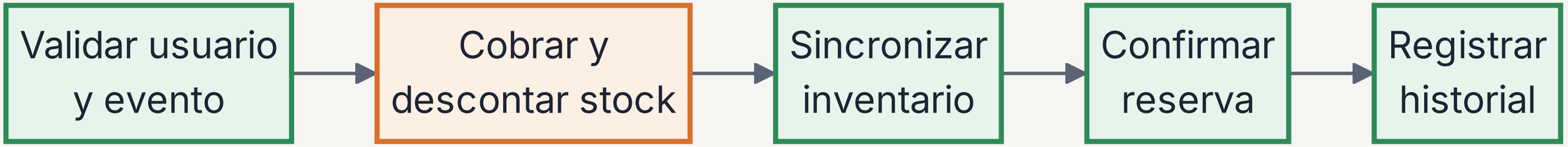
Un contenedor por base: MongoDB con una colección por servicio (cada uno toca solo la suya) · llamadas con reintentos y circuit breaker

Cada base, por lo que necesita su dato

Base	Qué guarda	Por qué	Alternativa descartada
MongoDB documental · NoSQL	Usuarios con historial, eventos con precios, reservas confirmadas	Modelo embebido: un documento trae todo sin joins; actualización atómica por documento; índices únicos	Relacional: joins para el historial y migraciones por cada campo nuevo
Redis clave-valor · NoSQL	Pago + stock por categoría (DB 0); caché de lecturas (DB 1)	Script Lua atómico: chequea, descuenta y cobra en una operación, < 1 ms, sin locks	Transacciones multi-documento de MongoDB: más lentas y complejas para un contador
PostgreSQL relacional	Event log append-only de cada SAGA y vistas de análisis	ACID + SQL + JSONB con índice GIN: auditoría consultable	Guardar la auditoría en MongoDB: sin SQL ni vistas para el análisis

Cassandra y DynamoDB se descartaron: más restrictivos o atados a un proveedor, sin ganancia a esta escala.

Una compra pasa por las tres



MongoDB

Datos del negocio

Fuente de verdad de usuarios, eventos y reservas. Lecturas disponibles vía caché.

Redis

Decide la venta

El Lua es el árbitro del stock: atómico. Además, caché de lecturas.

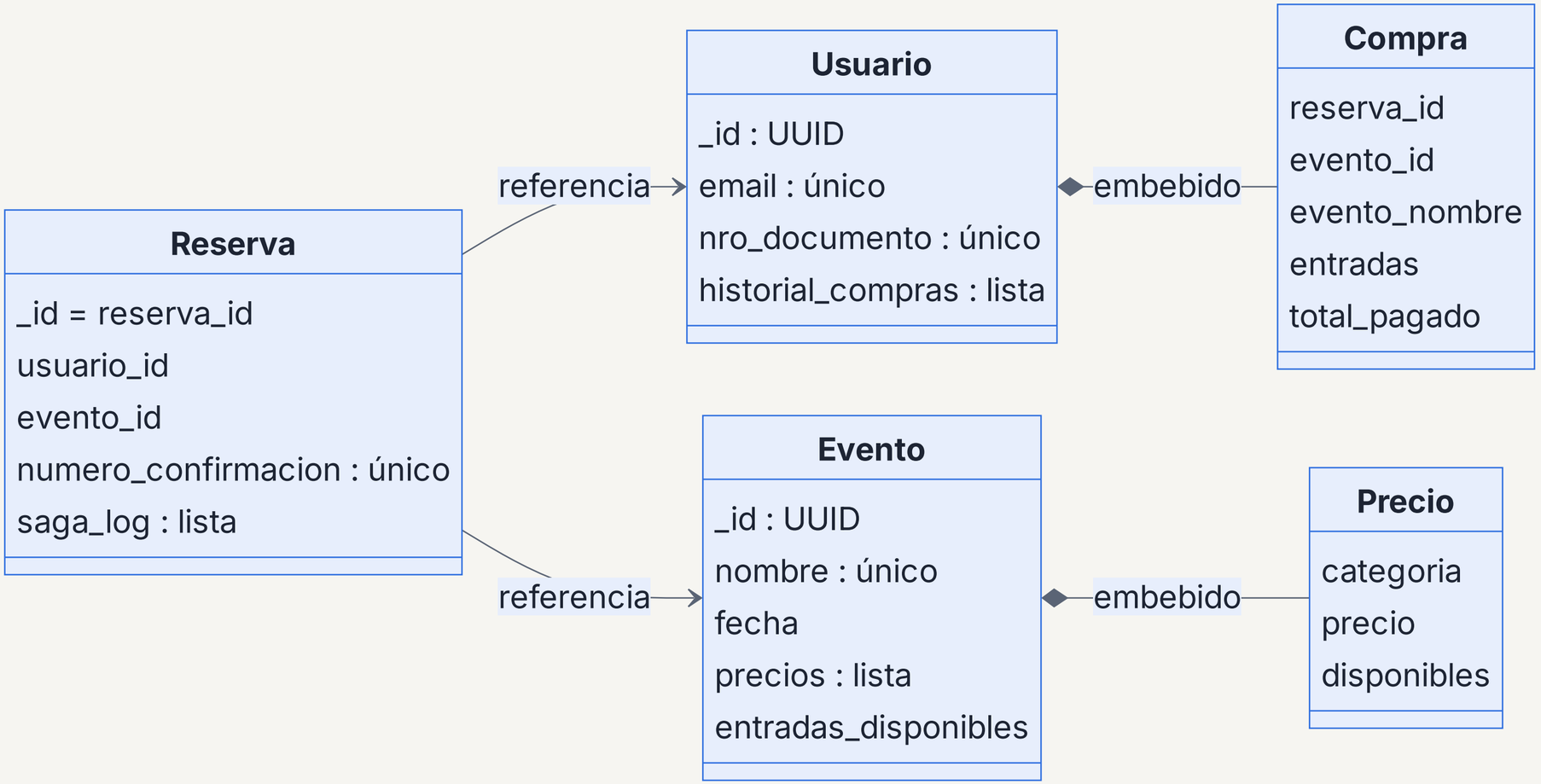
PostgreSQL

Registra todo

Cada paso, éxito o fallo, queda como evento. No bloquea la compra.

Cada paso de la compra también escribe su evento en PostgreSQL

Embebido dentro de un servicio, referencia entre servicios



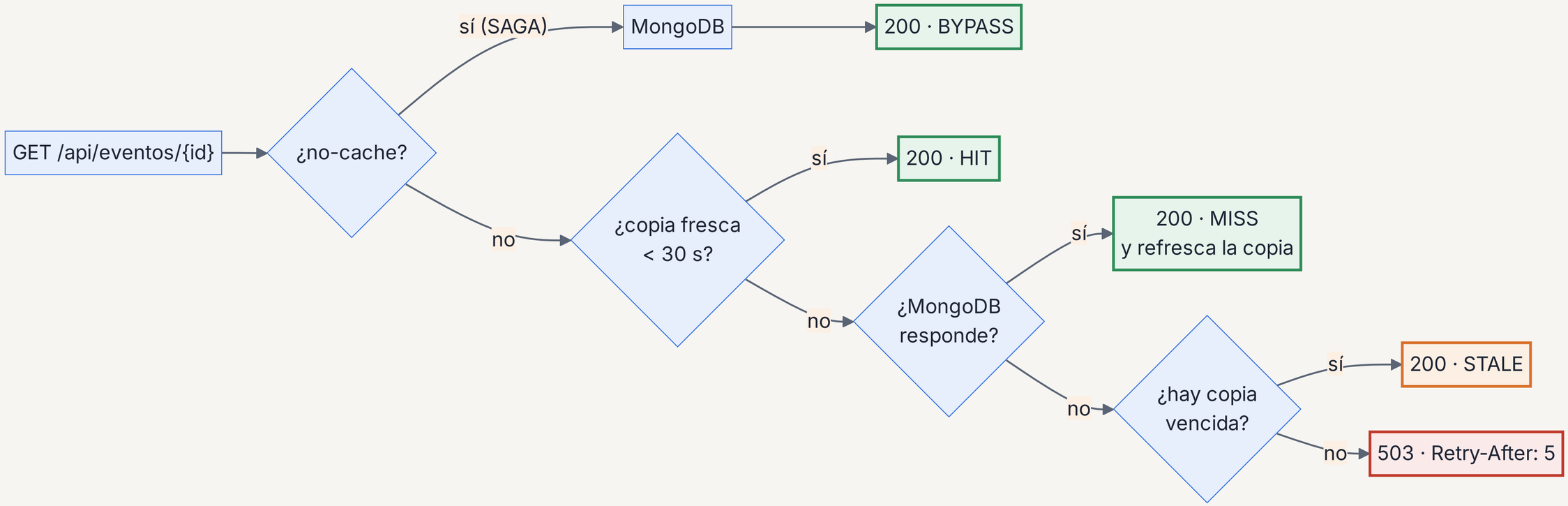
Embebido: se lee siempre junto y se actualiza atómicamente · Referencia: viven en servicios distintos

Cada operación elige su lado del teorema CAP

Operación	Prioriza	Cómo	Garantía
`GET /api/usuarios/{id}` `GET /api/eventos/{id}`	AP	Caché Redis *stale-if-error* delante de MongoDB	Responde aunque MongoDB caiga; el dato puede estar hasta 30 s viejo (más si MongoDB está caído)
`POST /api/reservar`	CP	Lua atómico + idempotencia + SAGA con compensación + `w=majority`	Nunca vende de más ni cobra dos veces; si no puede garantizarlo, responde error
Lectura dentro de la SAGA	CP	`Cache-Control: no-cache`	Nunca usa un dato vencido para decidir una compra

Honesto: cada base corre en un solo nodo. La read preference y `w=majority` están configurados, pero sin replica set no se ejercitan; la disponibilidad de lecturas se demuestra con la caché.

Caché stale-if-error: mejor un dato viejo que no responder



Seis handlers en serie: un paso o corta la cadena

1 ValidadorDeDatos
cantidad, método de pago, UUIDs

422 VALIDATION_ERROR

→

2 ValidadorInventario
GET usuario (no-cache)

404 USER_NOT_FOUND
503 Usuarios caído

→

3 ValidadorEvento
publicado · categoría · stock

404 · 409 · 422
503 Eventos caído

↓ continúa

4 ProcesadorPago
Lua en Redis + decrementar en Eventos

409 sin stock
503 → compensa Redis

→

5 ConfirmadorReserva
insert MongoDB + historial

500 → compensa Redis y Eventos

→

6 Auditor
SAGA_COMPLETED

no bloquea

Rojo: error sin nada que deshacer · Naranja: ya se modificó estado y el handler lo revierte · 201 si nadie corta

El patrón en el código

```python
# chain/handler.py
class BaseHandler(Handler):
    async def handle(self, ctx):
        if ctx.error:          # la cadena ya se cortó
            return ctx
        await self._process(ctx) # el paso propio
        if ctx.error:
            return ctx
        return await self._execute_next(ctx)

# chain/validators.py · ChainBuilder.build()
datos.set_next(usuario)
usuario.set_next(evento)
evento.set_next(pago)
pago.set_next(confirmacion)
confirmacion.set_next(auditor)
```

Una responsabilidad por handler

Validar, cobrar, confirmar y auditar están separados y se testean por separado.

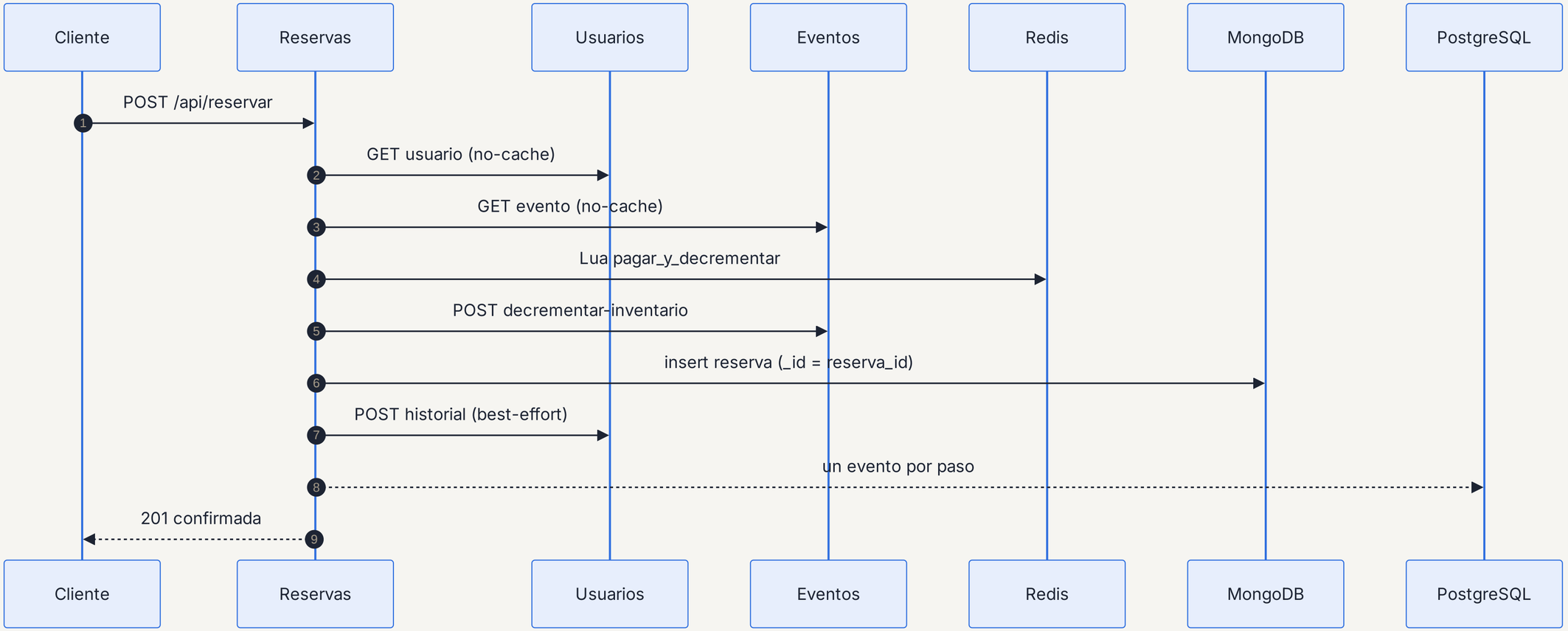
Corte temprano

Un error en los pasos 1-3 evita tocar Redis o MongoDB: no hay nada que compensar.

Compensación localizada

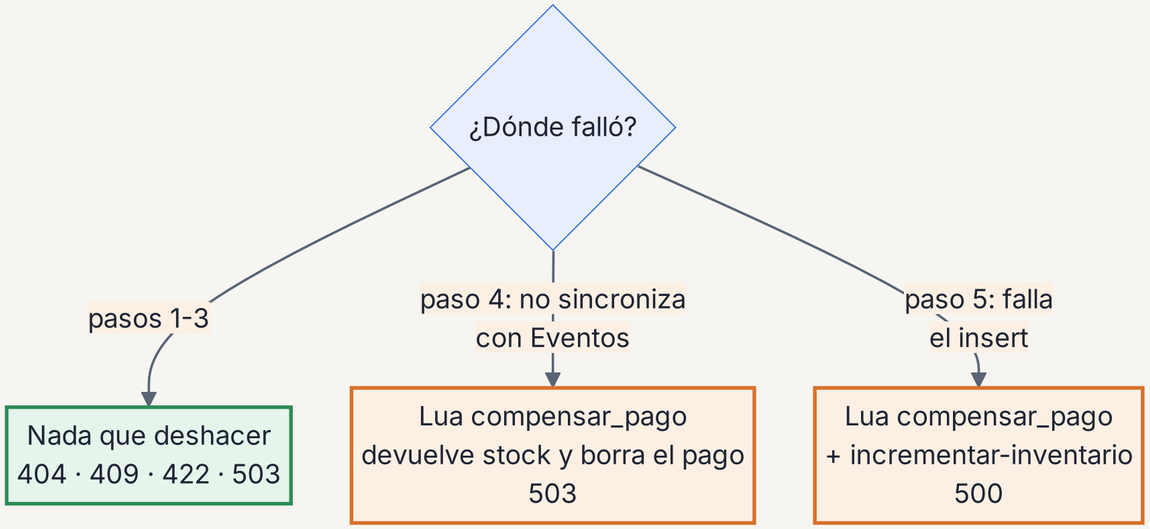
Solo los handlers 4 y 5 modifican estado, y solo ellos saben deshacerlo.

Reservas coordina la compra de punta a punta



Orquestación y no coreografía: un solo lugar conoce el orden y el estado, así que compensar y auditar es directo

Si algo falla después de cobrar, se deshace



Auditoría real de una compensación

inicio	SAGA_STARTED
+18 ms	USUARIO_VALIDADO
+36 ms	EVENTO_VALIDADO
+42 ms	PAGO_PROCESADO cobro real
+47 ms	COMPENSACION_EJECUTADA
+51 ms	SAGA_FAILED

Aforo antes y después: 5 → 5. Sin `INVENTARIO_DECREMENTADO`: Eventos nunca se enteró.

Compensaciones inline y sincrónicas dentro del handler que modificó el estado

No hay doble venta: el stock lo decide un script Lua

```lua
-- pagar_y_decrementar (Redis ejecuta un script por vez)
if redis.call('EXISTS', pago_key) == 1 then
  return {1, ya_pagado}        -- idempotente
end
redis.call('SETNX', inventario_key, seed)
local disponible = tonumber(redis.call('GET', inventario_key))
if disponible < cantidad then
  return {-2, disponible}      -- sin stock, nada cambia
end
redis.call('DECRBY', inventario_key, cantidad)
redis.call('HSET', pago_key, ...)  -- registra el pago
redis.call('EXPIRE', pago_key, ttl)
return {1, disponible - cantidad}
```

Prueba con JMeter

100 compradores × 3 intentos contra un evento con 50 entradas

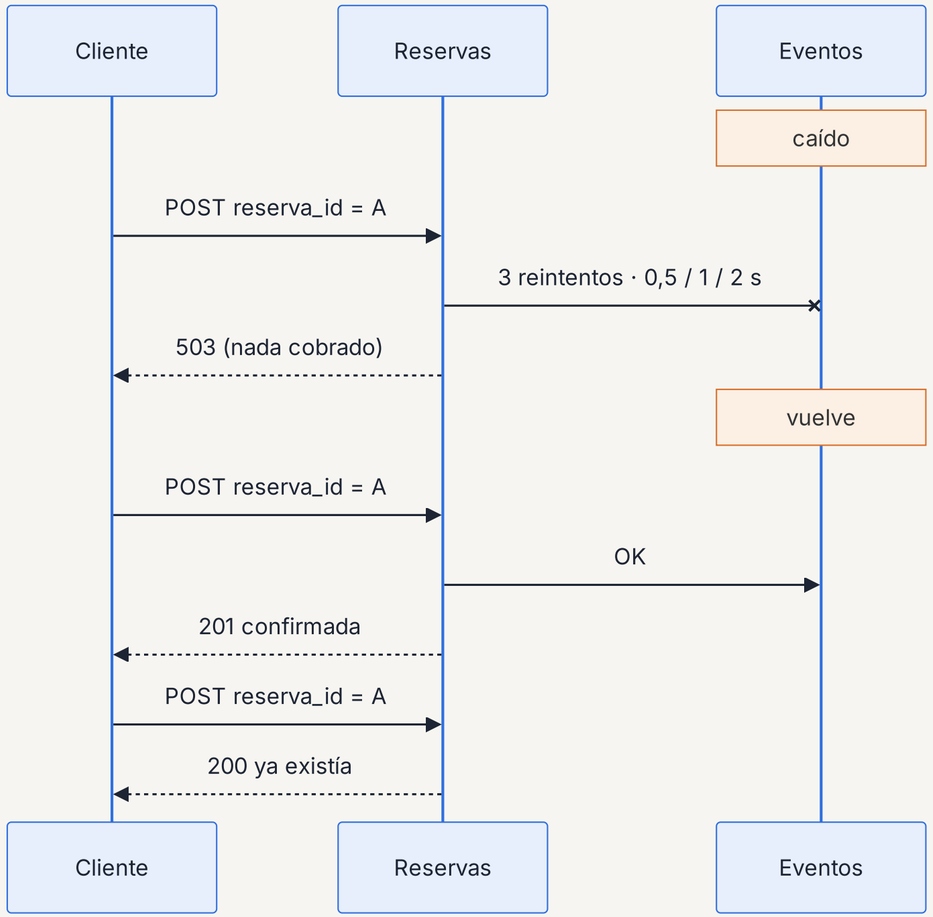
50
compras · 201

250
rechazos · 409

0
entradas al final · inventario exacto

Chequear y descontar no pueden intercalarse entre dos compradores

Reintentar siempre es seguro



Idempotencia por `reserva_id`

Ya confirmada en MongoDB → 200. El Lua no cobra dos veces. El historial usa `$ne`.

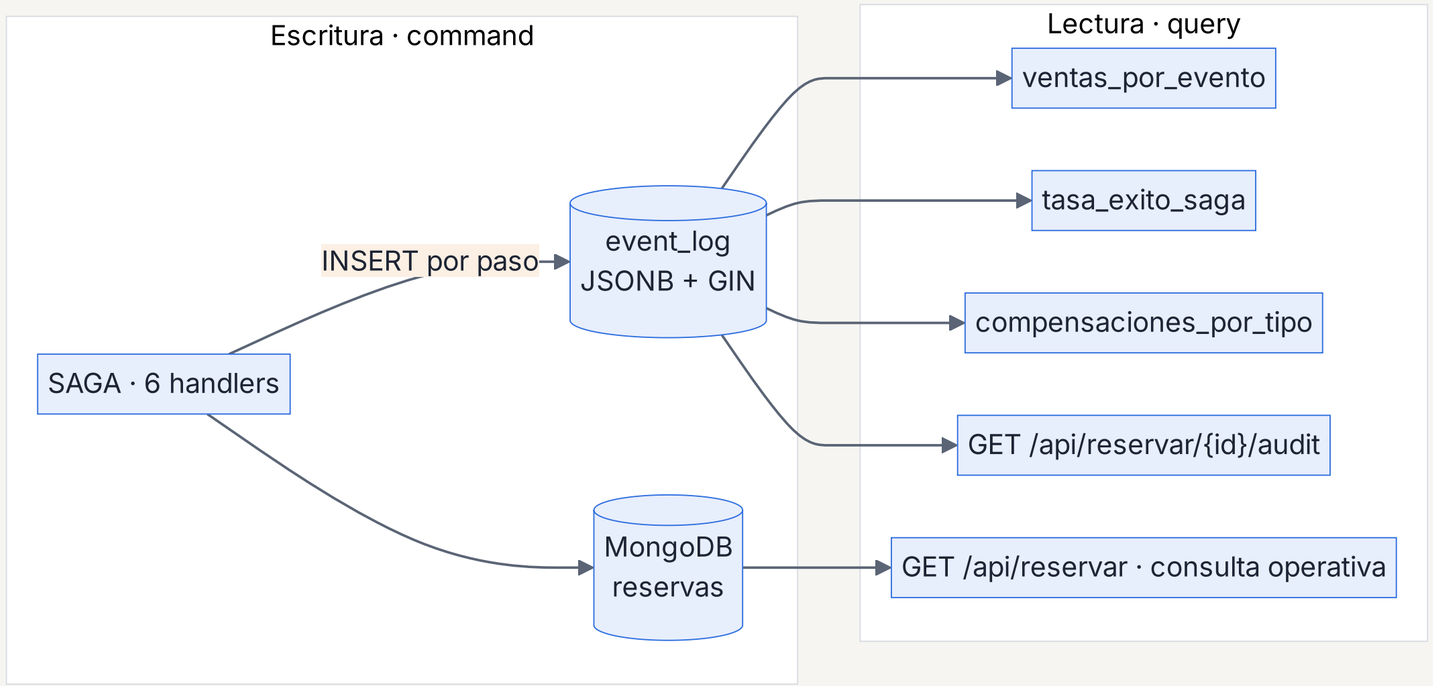
Reintentos con backoff

3 reintentos ante timeout o 5xx; un 404 no se reintenta.

Circuit breaker

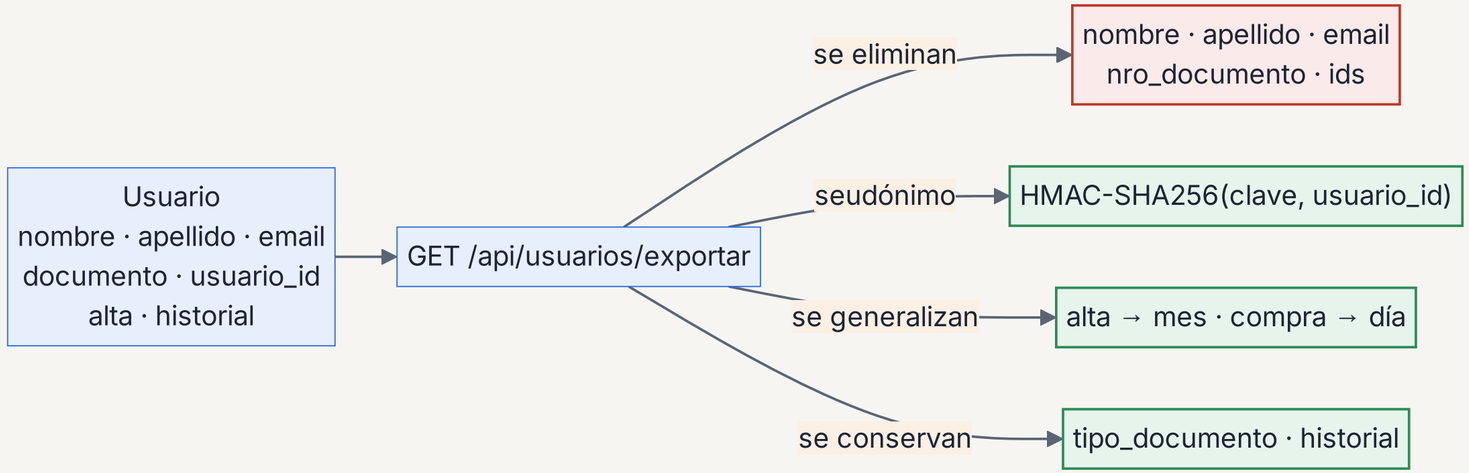
5 fallos seguidos → abierto 30 s, rechaza al instante; después prueba con una petición (half-open).

Event log append-only + CQRS



Event sourcing de auditoría: cada paso, incluidos fallos y compensaciones, queda como evento inmutable. El estado operativo vive en MongoDB (sin replay). Las ventas se cuentan por `RESERVA_CONFIRMADA`: un cobro compensado no las infla.

Export anonimizado: minimizar, no hashear



¿Por qué no un hash?

Un DNI o un nombre hasheado sin clave se revierte por diccionario en segundos.

Anonimizar vs seudonimizar

Sin clave configurada es efímera (anonimización); con clave fija, seudónimos estables.

JSON y CSV

El CSV neutraliza la inyección de fórmulas. Los datos personales viven en un único documento.

Qué está probado

261

tests en verde (`make test`): usuarios 56 · eventos 93 · reservas 112

0

ventas de más en 300 compras concurrentes contra 50 entradas

≈50

req/s de techo de la SAGA completa (objetivo: 100)

Escenario	Resultado verificado contra el stack
MongoDB caído	Lecturas siguen (`HIT` → `STALE`); la lectura estricta da 503
Eventos caído durante una compra	503 sin cobro; al volver, reintento → 201 → 200
Falla después de cobrar	Compensación: aforo igual antes y después, auditada

Lo que no está resuelto (y lo decimos)

Rendimiento

La SAGA completa satura en ≈ 50 req/s: no se cumple p99 < 1 s a 100 req/s. Descartados: CPU de Python y commit sincrónico de PostgreSQL. Sospechas: journal de MongoDB, proxy de Docker en WSL2.

Un nodo por base

Sin replica set ni réplica de Redis. Antes de agregar réplicas hay que ajustar la read preference (`maxStalenessSeconds ≥ 90`).

Event sourcing parcial

El log es de auditoría; el estado no se reconstruye desde los eventos.

Deuda técnica

Una segunda capa de compensación en el orquestador es redundante y registra un evento de más cuando falla el paso 5.

Privacidad

Sin endpoint de baja, sin cifrado de campos, sin k-anonimato.

Detalle en jmeter/README.md y brain/learnings/learnings.md

CIERRE

Consistencia fuerte donde se mueve plata. Disponibilidad en el catálogo.

SAGA orquestada sobre una Chain of Responsibility, Lua atómico para el stock, auditoría completa en PostgreSQL y lecturas que sobreviven a la caída de MongoDB.

`make up` `make test` `make jmeter` `python3 demo/servidor.py`

Demo en vivo a continuación